

CAHIER DES CHARGES TECHNIQUE

BACKEND — GlobalMarket

Plateforme E-Commerce Mondiale Multi-Vendeurs

Nom du projet : GlobalMarket — Backend API

Version : 1.0

Date : Juillet 2026

Auteur : Équipe Développement GlobalMarket

Statut : Document de référence

Périmètre du document

Ce cahier des charges technique couvre **l'intégralité du backend** de la plateforme GlobalMarket, incluant :

- Architecture serveur (Node.js / Express)
 - Base de données PostgreSQL
 - API REST complète (74 endpoints)
 - Authentification & autorisation (JWT, RBAC)
 - Intégrations paiement (Stripe, PayPal, Mobile Money)
 - Système de livraison & suivi
 - Notifications (email, push, SMS)
 - Sécurité & conformité RGPD
 - Déploiement & observabilité
-

Stack technique

Composant	Technologie	Version
Runtime	Node.js	20 LTS
Framework	Express.js	4.x
Base de données	PostgreSQL	16
ORM	Prisma	5.x
Cache	Redis	7.x
Auth	JWT (jsonwebtoken)	9.x
Validation	Zod	3.x
Upload	Multer + Sharp	—
Email	Nodemailer	6.x
PDF	PDFKit	4.x
Tests	Vitest + Supertest	—
Conteneurs	Docker + Docker Compose	—

Documents associés

Document	Chemin
Cahier des charges Frontend	<code>frontend/docs/CAHIER_DE_CHARGES_COMPLET.md</code>
PDF Frontend	<code>frontend/docs/CAHIER_DE_CHARGES_COMPLET.pdf</code>
Ce document (Backend)	<code>backend/docs/CAHIER_DE_CHARGES_BACKEND.md</code>

Chapitre 1 — Introduction & Contexte

1.1 Objectif du projet

GlobalMarket est une plateforme e-commerce mondiale multi-vendeurs visant à connecter acheteurs et vendeurs à l'échelle internationale. Le backend constitue le cœur système de la plateforme, responsable de la logique métier, de la gestion des données, de l'authentification et de l'intégration avec les services tiers.

1.2 Positionnement

Le backend doit :

- Servir une API RESTful consommée par le frontend React
- Gérer la logique métier complexe (multi-vendeurs, paiements escrow, livraison)
- Assurer la sécurité et la conformité RGPD
- Être scalable, résilient et observable
- Supporter le déploiement conteneurisé (Docker)

1.3 Principes directeurs

Principe	Description
Séparation des responsabilités	Controllers ↔ Services ↔ Repositories
API-First	Le frontend définit le contrat ; le backend l'implémente
Sécurité par conception	Validation de toutes les entrées, authentification systématique
Idempotence	Les opérations critiques (paiement, commande) sont idempotentes
Audit trail	Toutes les mutations importantes sont tracées
Fail-safe	Les erreurs retournent toujours <code>{ message: string }</code>

1.4 Contract frontend → backend

1.4.1 Format de réponse

```
// Succès (2xx)
{
  "data": { ... },
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 150,
    "totalPages": 8
  }
}

// Erreur (4xx / 5xx)
{
  "message": "Description de l'erreur en français"
}
```

1.4.2 Authentification

```
Authorization: Bearer <jwt_token>
```

1.4.3 Content-Type

- Requêtes : `application/json` (sauf uploads : `multipart/form-data`)
- Réponses : `application/json` (sauf factures : `application/pdf`)

1.4.4 Pagination

Toutes les listes supportent :

```
GET /api/resource?page=1&limit=20&sort=created_at&order=desc&search=terme
```

1.4.5 Proxys

Le frontend Vite proxy `/api` → `http://localhost:5000`

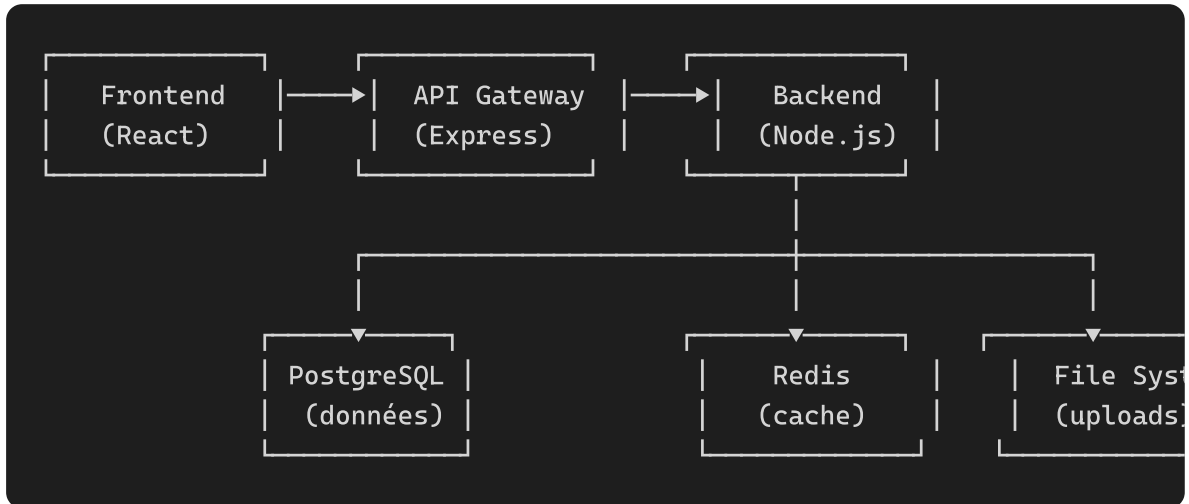
Toutes les routes backend commencent par `/api/`.

1.5 Portées fonctionnelles

Module	Endpoints	Priorité
Authentification	10	P0
Produits	14	P0
Catégories	4	P0
Panier	7	P0
Commandes	6	P0
Paielements	5	P0
Vendeur	14	P0
Administration	13	P1
Support	7	P1
Uploads	4	P0
Total	84	—

Chapitre 2 — Architecture Technique

2.1 Architecture générale



2.2 Structure du projet

```
backend/
├── src/
│   ├── config/                # Configuration (DB, Redis, JWT, mail, etc.)
│   │   ├── database.js
│   │   ├── redis.js
│   │   ├── jwt.js
│   │   ├── mail.js
│   │   └── upload.js
│   ├── middleware/            # Middlewares Express
│   │   ├── auth.js            # Vérification JWT
│   │   ├── role.js            # RBAC (admin, seller, buyer, delivery)
│   │   ├── validate.js        # Validation Zod
│   │   ├── upload.js          # Gestion multer
│   │   ├── rateLimiter.js     # Limitation de débit
│   │   └── errorHandler.js
│   ├── routes/                # Routeurs Express (un par module)
│   │   ├── auth.routes.js
│   │   ├── product.routes.js
│   │   ├── category.routes.js
│   │   ├── cart.routes.js
│   │   ├── order.routes.js
│   │   ├── payment.routes.js
│   │   ├── seller.routes.js
│   │   ├── admin.routes.js
│   │   ├── support.routes.js
│   │   └── upload.routes.js
│   ├── controllers/           # Controllers (requete → réponse)
│   │   ├── auth.controller.js
│   │   ├── product.controller.js
│   │   ├── category.controller.js
│   │   ├── cart.controller.js
│   │   ├── order.controller.js
│   │   ├── payment.controller.js
│   │   ├── seller.controller.js
│   │   ├── admin.controller.js
│   │   ├── support.controller.js
│   │   └── upload.controller.js
│   ├── services/              # Logique métier
│   │   ├── auth.service.js
│   │   ├── product.service.js
│   │   ├── category.service.js
│   │   ├── cart.service.js
│   │   ├── order.service.js
│   │   └── payment.service.js
```

```
| | | | | seller.service.js
| | | | | admin.service.js
| | | | | support.service.js
| | | | | upload.service.js
| | | | | email.service.js
| | | | | notification.service.js
| | | | | invoice.service.js
| | | | |
| | | | | repositories/          # Accès données (Prisma)
| | | | | | user.repository.js
| | | | | | product.repository.js
| | | | | | category.repository.js
| | | | | | cart.repository.js
| | | | | | order.repository.js
| | | | | | payment.repository.js
| | | | | | seller.repository.js
| | | | | | coupon.repository.js
| | | | | | support.repository.js
| | | | | | settings.repository.js
| | | | | |
| | | | | | validators/      # Schémas Zod
| | | | | | | auth.validator.js
| | | | | | | product.validator.js
| | | | | | | order.validator.js
| | | | | | | payment.validator.js
| | | | | | | seller.validator.js
| | | | | |
| | | | | | utils/          # Utilitaires
| | | | | | | slugify.js
| | | | | | | pagination.js
| | | | | | | hash.js
| | | | | | | token.js
| | | | | | | logger.js
| | | | | |
| | | | | | prisma/
| | | | | | | schema.prisma  # Schéma Prisma (source de vérité DB)
| | | | | |
| | | | | | app.js          # Point d'entrée Express
| | | | |
| | | | | prisma/
| | | | | | migrations/
| | | | |
| | | | | uploads/          # Fichiers uploadés (gitignored)
| | | | |
| | | | | tests/
| | | | | | unit/
| | | | | | integration/
| | | | | | fixtures/
| | | | |
| | | | | docker/
| | | | | | Dockerfile
| | | | | | docker-compose.yml
| | | | |
| | | | | .env.example
| | | | | package.json
| | | | | vitest.config.js
```


2.3 Pattern architectural : Controller → Service → Repository

```
Request → Controller → Service → Repository → Prisma → PostgreSQL  
      ↓  
    Validation (Zod)  
      ↓  
Auth middleware (JWT)  
      ↓  
Role middleware (RBAC)
```

Exemple flux complet

```
POST /api/seller/products  
├── authMiddleware → vérifie JWT, injecte req.user  
├── sellerMiddleware → vérifie req.user.role ≡ 'seller'  
├── validate(productSchema) → valide le body avec Zod  
└── sellerController.createProduct  
    ├── sellerService.createProduct  
    │   ├── productRepository.create(data)  
    │   │   └── prisma.product.create({ ... })  
    │   └── Retourne { data: product }  
    └──
```

2.4 Configuration

Variables d'environnement (.env)

```
# Serveur
PORT=5000
NODE_ENV=development

# PostgreSQL
DATABASE_URL=postgresql://globalmarket:secret@localhost:5432/globalmarket

# Redis
REDIS_URL=redis://localhost:6379

# JWT
JWT_SECRET=votre-secret-jwt-tres-long
JWT_EXPIRES_IN=7d
JWT_REFRESH_SECRET=votre-refresh-secret-jwt
JWT_REFRESH_EXPIRES_IN=30d

# Stripe
STRIPE_SECRET_KEY=sk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...

# PayPal
PAYPAL_CLIENT_ID=...
PAYPAL_CLIENT_SECRET=...

# Email (SMTP)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=noreply@globalmarket.com
SMTP_PASS=...
EMAIL_FROM=GlobalMarket <noreply@globalmarket.com>

# Upload
UPLOAD_DIR=./uploads
MAX_FILE_SIZE=10485760

# Frontend URL (pour emails)
FRONTEND_URL=http://localhost:5173
```

2.5 Base de données : PostgreSQL 16

Pourquoi PostgreSQL

Critère	PostgreSQL
ACID	Transactionnel, fiable
JSON	Colonnes JSONB pour données flexibles
Full-text search	Recherche texte intégrée
Extensions	pg_trgm (fuzzy search), uuid-ossdp
Performance	Index B-tree, GIN, GiST
Écosystème	Prisma ORM, mature et documenté

Configuration PostgreSQL

```
-- Extension pour recherche fuzzy
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Extension pour UUID
CREATE EXTENSION IF NOT EXISTS "uuid-ossdp";

-- Locale français
SET lc_messages = 'fr_FR.UTF-8';
```

2.6 Cache Redis

Usage	TTL	Clé
Session utilisateur	7d	user:{id}
Produit en vedette	1h	product:featured
Catégories (arbre)	24h	categories:tree
Recherche populaire	6h	search:popular
Rate limiting	1min	ratelimit:{ip}:{route}
Panier utilisateur	30d	cart:{userId}

2.7 Logging & Monitoring

Outil	Usage
Winston	Structured logging (JSON)
Morgan	HTTP request logging
Prometheus (optionnel)	Métriques
Sentry (optionnel)	Error tracking

Niveaux de log

Niveau	Usage
error	Erreurs serveur, exceptions
warn	Rate limiting, dépréciations
info	Requêtes réussies, événements métier
debug	SQL queries, payloads (dev uniquement)

Chapitre 3 — Base de Données PostgreSQL

3.1 Schéma Prisma (source de vérité)

Le schéma Prisma définit l'intégralité des tables, relations, index et contraintes.

```
// prisma/schema.prisma

generator client {
  provider      = "prisma-client-js"
  previewFeatures = ["fullTextSearch"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

3.2 Enumérations

```
enum UserRole {
    buyer
    seller
    admin
    delivery
}

enum UserStatus {
    active
    inactive
    suspended
    banned
}

enum OrderStatus {
    pending
    confirmed
    processing
    shipped
    delivered
    cancelled
    refunded
}

enum PaymentStatus {
    pending
    processing
    completed
    failed
    refunded
    partially_refunded
}

enum PaymentMethod {
    credit_card
    debit_card
    paypal
    apple_pay
    google_pay
    stripe
    mobile_money
    bank_transfer
    cash_on_delivery
}
```

```
enum ProductStatus {
    draft
    active
    inactive
    archived
    suspended
}

enum SellerPlan {
    basic
    pro
    enterprise
}

enum TicketStatus {
    open
    in_progress
    waiting
    resolved
    closed
}

enum TicketPriority {
    low
    medium
    high
    urgent
}

enum NotificationType {
    order
    payment
    shipping
    promotion
    system
    support
}
```

3.3 Tables

3.3.1 Utilisateurs

```
model User {
  id          String      @id @default(uuid()) @db.Uuid
  email       String      @unique @db.VarChar(255)
  name        String      @db.VarChar(255)
  passwordHash String      @map("password_hash") @db.VarChar(255)
  phone       String?     @db.VarChar(20)
  avatar      String?     @db.VarChar(500)
  role        UserRole     @default(buyer)
  status      UserStatus  @default(active)
  emailVerified Boolean    @default(false) @map("email_verified")
  lastLoginAt DateTime?   @map("last_login_at")
  createdAt   DateTime     @default(now()) @map("created_at")
  updatedAt   DateTime     @updatedAt @map("updated_at")

  // Relations
  addresses      Address[]
  seller         Seller?
  orders         Order[]      @relation("BuyerOrders")
  cart           Cart?
  wishlist       WishlistItem[]
  reviews        Review[]
  supportTickets SupportTicket[]
  notifications  Notification[]
  refreshTokens  RefreshToken[]

  @@map("users")
}

model RefreshToken {
  id          String      @id @default(uuid()) @db.Uuid
  token       String      @unique @db.Text
  userId      String      @map("user_id") @db.Uuid
  expiresAt   DateTime     @map("expires_at")
  createdAt   DateTime     @default(now()) @map("created_at")

  user        User        @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@map("refresh_tokens")
}
```


3.3.2 Adresses

```
model Address {
  id          String @id @default(uuid()) @db.Uuid
  userId      String @map("user_id") @db.Uuid
  label       String @db.VarChar(100) // "Domicile", "Bureau", etc.
  firstName   String @map("first_name") @db.VarChar(100)
  lastName    String @map("last_name") @db.VarChar(100)
  address1    String @db.VarChar(255)
  address2    String? @db.VarChar(255)
  city        String @db.VarChar(100)
  state       String? @db.VarChar(100)
  postalCode  String @map("postal_code") @db.VarChar(20)
  country     String @db.VarChar(100)
  phone       String? @db.VarChar(20)
  isDefault   Boolean @default(false) @map("is_default")
  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @updatedAt @map("updated_at")

  user        User @relation(fields: [userId], references: [id], onDelete: Cascade)
  orders      Order[]

  @@index([userId])
  @@map("addresses")
}
```

3.3.3 Vendeurs (Seller Profile)

```
model Seller {
  id          String      @id @default(uuid()) @db.Uuid
  userId      String      @unique @map("user_id") @db.Uuid
  shopName    String      @map("shop_name") @db.VarChar(255)
  slug        String      @unique @db.VarChar(255)
  description  String?     @db.Text
  logo        String?     @db.VarChar(500)
  banner      String?     @db.VarChar(500)
  category    String      @db.VarChar(100)
  country     String      @db.VarChar(100)
  businessType String      @default("individual") @map("business_type")
  plan        SellerPlan  @default(basic)
  commissionRate Decimal  @default(0.12) @map("commission_rate") @db.Decimal(3, 2)
  rating       Decimal    @default(0) @db.Decimal(3, 2)
  totalSales   Int        @default(0) @map("total_sales")
  totalRevenue Decimal    @default(0) @map("total_revenue") @db.Decimal(10, 2)
  verified     Boolean    @default(false)
  active       Boolean    @default(true)
  bankAccountJson Json?    @map("bank_account") // { iban, bic, holderName }
  createdAt    DateTime   @default(now()) @map("created_at")
  updatedAt    DateTime   @updatedAt @map("updated_at")

  user      User      @relation(fields: [userId], references: [id])
  products  Product[]
  orders    OrderItem[] @relation("SellerOrders")
  payouts   Payout[]

  @@index([userId])
  @@index([slug])
  @@index([category])
  @@map("sellers")
}
```

3.3.4 Catégories

```
model Category {
  id          String      @id @default(uuid()) @db.Uuid
  name        String      @db.VarChar(100)
  slug        String      @unique @db.VarChar(100)
  description String?      @db.Text
  icon        String?      @db.VarChar(50)
  image       String?      @db.VarChar(500)
  parentId    String?      @map("parent_id") @db.Uuid
  sortOrder   Int          @default(0) @map("sort_order")
  active       Boolean      @default(true)
  createdAt   DateTime      @default(now()) @map("created_at")
  updatedAt   DateTime      @updatedAt @map("updated_at")

  parent      Category?    @relation("CategoryTree", fields: [parentId], re-
  children    Category[]   @relation("CategoryTree")
  products    Product[]

  @@index([parentId])
  @@index([slug])
  @@map("categories")
}
```

3.3.5 Produits

```
model Product {
  id          String          @id @default(uuid()) @db.Uuid
  sellerId    String          @map("seller_id") @db.Uuid
  categoryId  String          @map("category_id") @db.Uuid
  name        String          @db.VarChar(255)
  slug        String          @unique @db.VarChar(255)
  description  String          @db.Text
  shortDescription String?    @map("short_description") @db.VarChar(500)
  price       Decimal         @db.Decimal(10, 2)
  compareAtPrice Decimal?    @map("compare_at_price") @db.Decimal(10, 2)
  costPrice   Decimal?        @map("cost_price") @db.Decimal(10, 2)
  sku         String?         @unique @db.VarChar(100)
  barcode     String?         @db.VarChar(100)
  stock       Int             @default(0)
  lowStockThreshold Int       @default(5) @map("low_stock_threshold")
  weight      Decimal?        @db.Decimal(8, 2) // en grammes
  dimensions  Json?           // { length, width, height }
  status      ProductStatus   @default(draft)
  featured    Boolean         @default(false)
  trending    Boolean         @default(false)
  averageRating Decimal       @default(0) @map("average_rating") @db.Decimal(3, 2)
  reviewCount Int             @default(0) @map("review_count")
  salesCount  Int             @default(0) @map("sales_count")
  viewCount   Int             @default(0) @map("view_count")
  tags        String[]        @db.VarChar(50)
  metaTitle   String?         @map("meta_title") @db.VarChar(255)
  metaDescription String?     @map("meta_description") @db.VarChar(500)
  publishedAt DateTime?       @map("published_at")
  createdAt   DateTime        @default(now()) @map("created_at")
  updatedAt   DateTime        @updatedAt @map("updated_at")

  seller      Seller          @relation(fields: [sellerId], references: [sellerId])
  category    Category        @relation(fields: [categoryId], references: [categoryId])
  images      ProductImage[]
  variants    ProductVariant[]
  orderItems  OrderItem[]
  reviews     Review[]
  wishlistItems WishlistItem[]
  cartItems   CartItem[]

  @@index([sellerId])
  @@index([categoryId])
  @@index([status])
  @@index([featured])
  @@index([createdAt])
}
```

```
    @@index([price])
    @@index([name])
    @@map("products")
}

model ProductImage {
    id          String  @id @default(uuid()) @db.Uuid
    productId   String  @map("product_id") @db.Uuid
    url         String  @db.VarChar(500)
    alt         String? @db.VarChar(255)
    sortOrder   Int      @default(0) @map("sort_order")
    isPrimary   Boolean @default(false) @map("is_primary")
    createdAt   DateTime @default(now()) @map("created_at")

    product     Product @relation(fields: [productId], references: [id], onDelete="cascade")

    @@index([productId])
    @@map("product_images")
}

model ProductVariant {
    id          String  @id @default(uuid()) @db.Uuid
    productId   String  @map("product_id") @db.Uuid
    name        String  @db.VarChar(100)    // "Rouge / XL"
    sku         String? @unique @db.VarChar(100)
    price       Decimal @db.Decimal(10, 2)
    stock       Int      @default(0)
    attributes  Json?    // { color: "rouge", size: "XL" }
    createdAt   DateTime @default(now()) @map("created_at")

    product     Product @relation(fields: [productId], references: [id], onDelete="cascade")
    cartItems   CartItem[]
    orderItems  OrderItem[]

    @@index([productId])
    @@map("product_variants")
}
```

3.3.6 Panier

```
model Cart {
  id          String      @id @default(uuid()) @db.Uuid
  userId      String      @unique @map("user_id") @db.Uuid
  couponId    String?     @map("coupon_id") @db.Uuid
  createdAt   DateTime    @default(now()) @map("created_at")
  updatedAt   DateTime    @updatedAt @map("updated_at")

  user        User        @relation(fields: [userId], references: [id], onDelete: Cascade)
  coupon      Coupon?     @relation(fields: [couponId], references: [id])
  items       CartItem[]

  @@map("carts")
}

model CartItem {
  id          String      @id @default(uuid()) @db.Uuid
  cartId      String      @map("cart_id") @db.Uuid
  productId   String      @map("product_id") @db.Uuid
  variantId   String?     @map("variant_id") @db.Uuid
  quantity    Int         @default(1)
  createdAt   DateTime    @default(now()) @map("created_at")
  updatedAt   DateTime    @updatedAt @map("updated_at")

  cart        Cart        @relation(fields: [cartId], references: [id], onDelete: Cascade)
  product     Product     @relation(fields: [productId], references: [id], onDelete: Cascade)
  variant     ProductVariant? @relation(fields: [variantId], references: [id], onDelete: Cascade)

  @@unique([cartId, productId, variantId])
  @@map("cart_items")
}
```

3.3.7 Coupons

```
model Coupon {
  id          String    @id @default(uuid()) @db.Uuid
  code        String    @unique @db.VarChar(50)
  description  String?   @db.VarChar(255)
  discountType String    @map("discount_type") @db.VarChar(20) // "percent
  discountValue Decimal  @map("discount_value") @db.Decimal(10, 2)
  minAmount   Decimal?  @map("min_amount") @db.Decimal(10, 2)
  maxDiscount Decimal?  @map("max_discount") @db.Decimal(10, 2)
  usageLimit   Int?     @map("usage_limit")
  usedCount    Int       @default(0) @map("used_count")
  startDate    DateTime  @map("start_date")
  endDate      DateTime  @map("end_date")
  active        Boolean  @default(true)
  createdAt    DateTime  @default(now()) @map("created_at")

  carts        Cart[]
  orders        Order[]

  @@index([code])
  @@map("coupons")
}
```

3.3.8 Commandes

```
model Order {
    id                String                @id @default(uuid()) @db.Uuid
    orderNumber       String                @unique @map("order_number") @db.VarChar(255)
    buyerId           String                @map("buyer_id") @db.Uuid
    addressId         String                @map("address_id") @db.Uuid
    couponId          String?              @map("coupon_id") @db.Uuid
    status            OrderStatus @default(pending)
    subtotal          Decimal               @db.Decimal(10, 2)
    shippingCost      Decimal               @default(0) @map("shipping_cost") @db.Decimal(10, 2)
    taxAmount         Decimal               @default(0) @map("tax_amount") @db.Decimal(10, 2)
    discountAmount    Decimal               @default(0) @map("discount_amount") @db.Decimal(10, 2)
    totalAmount       Decimal               @map("total_amount") @db.Decimal(10, 2)
    currency          String                @default("EUR") @db.VarChar(3)
    notes             String?              @db.Text
    shippingMethod    String?              @map("shipping_method") @db.VarChar(50)
    trackingNumber     String?              @map("tracking_number") @db.VarChar(100)
    shippedAt         DateTime?             @map("shipped_at")
    deliveredAt       DateTime?             @map("delivered_at")
    cancelledAt       DateTime?             @map("cancelled_at")
    cancelReason      String?              @map("cancel_reason") @db.Text
    createdAt         DateTime              @default(now()) @map("created_at")
    updatedAt         DateTime              @updatedAt @map("updated_at")

    buyer            User                  @relation("BuyerOrders", fields: [buyerId], references: [id])
    address          Address                @relation(fields: [addressId], references: [id])
    coupon            Coupon?              @relation(fields: [couponId], references: [id])
    items            OrderItem[]
    payments          Payment[]
    shipments         Shipment[]

    @@index([buyerId])
    @@index([status])
    @@index([createdAt])
    @@index([orderNumber])
    @@map("orders")
}

model OrderItem {
    id                String                @id @default(uuid()) @db.Uuid
    orderId           String                @map("order_id") @db.Uuid
    productId         String                @map("product_id") @db.Uuid
    variantId         String?              @map("variant_id") @db.Uuid
    sellerId          String                @map("seller_id") @db.Uuid
    productName       String                @map("product_name") @db.VarChar(255)
    productImage      String?              @map("product_image") @db.VarChar(500)
```



```
quantity      Int
unitPrice     Decimal      @map("unit_price") @db.Decimal(10, 2)
totalPrice    Decimal      @map("total_price") @db.Decimal(10, 2)
commission     Decimal      @default(0) @db.Decimal(10, 2)
status        OrderStatus  @default(pending)
createdAt     DateTime      @default(now()) @map("created_at")

order         Order         @relation(fields: [orderId], references: [id])
product       Product       @relation(fields: [productId], references: [id])
variant       ProductVariant? @relation(fields: [variantId], references: [id])
seller        Seller        @relation("SellerOrders", fields: [sellerId], references: [id])

@@index([orderId])
@@index([sellerId])
@@index([productId])
@@map("order_items")
}
```

3.3.9 Paiements

```
model Payment {
  id          String        @id @default(uuid()) @db.Uuid
  orderId     String        @map("order_id") @db.Uuid
  amount      Decimal       @db.Decimal(10, 2)
  currency    String        @default("EUR") @db.VarChar(3)
  method      PaymentMethod
  status      PaymentStatus @default(pending)
  stripePaymentId String?   @map("stripe_payment_id") @db.VarChar(255)
  paypalOrderId String?    @map("paypal_order_id") @db.VarChar(255)
  mobileMoneyRef String?   @map("mobile_money_ref") @db.VarChar(255)
  refundAmount Decimal      @default(0) @map("refund_amount") @db.Decimal(10, 2)
  refundReason String?     @map("refund_reason") @db.Text
  metadata    Json?
  paidAt      DateTime?    @map("paid_at")
  createdAt   DateTime      @default(now()) @map("created_at")
  updatedAt   DateTime      @updatedAt @map("updated_at")

  order       Order         @relation(fields: [orderId], references: [id])

  @@index([orderId])
  @@index([stripePaymentId])
  @@index([status])
  @@map("payments")
}
```

3.3.10 Expéditions

```
model Shipment {
  id          String    @id @default(uuid()) @db.Uuid
  orderId     String    @map("order_id") @db.Uuid
  carrier     String    @db.VarChar(100)    // "DHL", "FedEx", "La Poste"
  trackingNumber String? @map("tracking_number") @db.VarChar(255)
  trackingUrl String?   @map("tracking_url") @db.VarChar(500)
  status      String    @default("preparing") @db.VarChar(50)
  estimatedDelivery DateTime? @map("estimated_delivery")
  shippedAt    DateTime? @map("shipped_at")
  deliveredAt DateTime? @map("delivered_at")
  weight      Decimal?  @db.Decimal(8, 2)
  cost        Decimal?  @db.Decimal(10, 2)
  createdAt   DateTime  @default(now()) @map("created_at")
  updatedAt   DateTime  @updatedAt @map("updated_at")

  order       Order     @relation(fields: [orderId], references: [id])

  @@index([orderId])
  @@index([trackingNumber])
  @@map("shipments")
}
```

3.3.11 Virements vendeurs (Payouts)

```
model Payout {
  id          String    @id @default(uuid()) @db.Uuid
  sellerId    String    @map("seller_id") @db.Uuid
  amount      Decimal   @db.Decimal(10, 2)
  currency    String    @default("EUR") @db.VarChar(3)
  status      String    @default("pending") @db.VarChar(20) // pending, processed
  method      String    @db.VarChar(50)    // bank_transfer, paypal
  reference   String?   @db.VarChar(255)
  notes       String?   @db.Text
  paidAt      DateTime? @map("paid_at")
  createdAt   DateTime  @default(now()) @map("created_at")

  seller      Seller    @relation(fields: [sellerId], references: [id])

  @@index([sellerId])
  @@index([status])
  @@map("payouts")
}
```

3.3.12 Avis (Reviews)

```
model Review {
  id          String   @id @default(uuid()) @db.Uuid
  productId   String   @map("product_id") @db.Uuid
  userId      String   @map("user_id") @db.Uuid
  rating      Int       @db.Int // 1-5
  title       String?  @db.VarChar(255)
  comment     String?  @db.Text
  images      String[] @db.VarChar(500)
  helpful     Int       @default(0)
  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @updatedAt @map("updated_at")

  product     Product @relation(fields: [productId], references: [id], onDelete: Cascade)
  user        User    @relation(fields: [userId], references: [id])

  @@unique([productId, userId])
  @@index([productId])
  @@index([userId])
  @@map("reviews")
}
```

3.3.13 Wishlist

```
model WishlistItem {
  id          String   @id @default(uuid()) @db.Uuid
  userId      String   @map("user_id") @db.Uuid
  productId   String   @map("product_id") @db.Uuid
  createdAt   DateTime @default(now()) @map("created_at")

  user        User    @relation(fields: [userId], references: [id], onDelete: Cascade)
  product     Product @relation(fields: [productId], references: [id], onDelete: Cascade)

  @@unique([userId, productId])
  @@map("wishlist_items")
}
```

3.3.14 Support (Tickets)

```
model SupportTicket {
  id          String          @id @default(uuid()) @db.Uuid
  userId      String          @map("user_id") @db.Uuid
  subject     String          @db.VarChar(255)
  category    String          @db.VarChar(50) // "order", "product", "account"
  priority    TicketPriority @default(medium)
  status      TicketStatus @default(open)
  orderId     String?         @map("order_id") @db.Uuid
  createdAt   DateTime        @default(now()) @map("created_at")
  updatedAt   DateTime        @updatedAt @map("updated_at")

  user        User            @relation(fields: [userId], references: [id])
  messages    TicketMessage[]

  @@index([userId])
  @@index([status])
  @@map("support_tickets")
}

model TicketMessage {
  id          String          @id @default(uuid()) @db.Uuid
  ticketId    String          @map("ticket_id") @db.Uuid
  senderId    String          @map("sender_id") @db.Uuid
  message     String          @db.Text
  attachments String[] @db.VarChar(500)
  isInternal  Boolean @default(false) @map("is_internal")
  createdAt   DateTime        @default(now()) @map("created_at")

  ticket      SupportTicket @relation(fields: [ticketId], references: [id],
  sender      User           @relation(fields: [senderId], references: [id])

  @@index([ticketId])
  @@map("ticket_messages")
}
```

3.3.15 Notifications

```
model Notification {
  id          String          @id @default(uuid()) @db.Uuid
  userId      String          @map("user_id") @db.Uuid
  type        NotificationType
  title       String          @db.VarChar(255)
  message     String          @db.Text
  data        Json?           // données contextuelles (orderId, etc.)
  read        Boolean         @default(false)
  createdAt   DateTime        @default(now()) @map("created_at")

  user        User            @relation(fields: [userId], references: [id])

  @@index([userId, read])
  @@index([createdAt])
  @@map("notifications")
}
```

3.3.16 Paramètres système

```
model SystemSetting {
  id          String          @id @default(uuid()) @db.Uuid
  key         String          @unique @db.VarChar(100)
  value       Json
  updatedAt   DateTime        @updatedAt @map("updated_at")

  @@map("system_settings")
}
```

3.3.17 Fichiers uploadés

```
model UploadedFile {
  id          String   @id @default(uuid()) @db.Uuid
  filename    String   @db.VarChar(255)
  originalName String   @map("original_name") @db.VarChar(255)
  mimetype    String   @db.VarChar(100)
  size        Int      // en octets
  path        String   @db.VarChar(500)
  uploaderId  String?   @map("uploader_id") @db.Uuid
  createdAt   DateTime @default(now()) @map("created_at")

  @@index([uploaderId])
  @@map("uploaded_files")
}
```

3.4 Index de performance

```
-- Recherche produits par texte (GIN index)
CREATE INDEX idx_products_search ON products
  USING GIN (to_tsvector('french', name || ' ' || COALESCE(description, '')))

-- Recherche fuzzy sur les noms de produits
CREATE INDEX idx_products_name_trgm ON products
  USING GIN (name gin_trgm_ops);

-- Recherche par fourchette de prix
CREATE INDEX idx_products_price ON products (price) WHERE status = 'active';

-- Commandes par statut et date (dashboard admin)
CREATE INDEX idx_orders_status_date ON orders (status, created_at DESC);

-- Notifications non lues par utilisateur
CREATE INDEX idx_notifications_unread ON notifications (user_id, created_at)
  WHERE read = false;
```

3.5 Données de seed

Catégories par défaut

```
[
  { "name": "Électronique", "slug": "electronique", "icon": "📱" },
  { "name": "Mode & Vêtements", "slug": "mode-vetements", "icon": "👕" },
  { "name": "Maison & Jardin", "slug": "maison-jardin", "icon": "🏠" },
  { "name": "Sports & Loisirs", "slug": "sports-loisirs", "icon": "⚽" },
  { "name": "Beauté & Santé", "slug": "beaute-sante", "icon": "💄" },
  { "name": "Jouets & Enfants", "slug": "jouets-enfants", "icon": "🧸" },
  { "name": "Automobile", "slug": "automobile", "icon": "🚗" },
  { "name": "Livres & Médias", "slug": "livres-medias", "icon": "📖" },
  { "name": "Alimentation", "slug": "alimentation", "icon": "🍏" },
  { "name": "Art & Artisanat", "slug": "art-artisanat", "icon": "🎨" }
]
```

Admin par défaut

```
{
  "email": "admin@globalmarket.com",
  "name": "Administrateur",
  "password": "Admin@GlobalMarket2026!",
  "role": "admin"
}
```

Chapitre 4 — API REST :

Authentification

Base URL : `/api/auth`

4.1 POST /api/auth/login

Description : Connexion utilisateur

Auth : Non

Body :

```
{
  "email": "string (email valide)",
  "password": "string (min 8 caractères)"
}
```

Réponse 200 :

```
{
  "data": {
    "user": {
      "id": "uuid",
      "email": "string",
      "name": "string",
      "role": "buyer | seller | admin | delivery",
      "avatar": "string | null",
      "emailVerified": false
    },
    "token": "jwt_token",
    "refreshToken": "jwt_refresh_token"
  }
}
```

Erreurs :

- 400: "Email et mot de passe requis"
- 401: "Email ou mot de passe incorrect"
- 403: "Compte suspendu"

4.2 POST /api/auth/register

Description : Inscription nouvel utilisateur

Auth : Non

Body :


```
{
  "name": "string (2-255 caractères)",
  "email": "string (email valide)",
  "password": "string (min 8, 1 majuscule, 1 chiffre)"
}
```

Réponse 201 :

```
{
  "data": {
    "user": { "id": "...", "email": "...", "name": "...", "role": "buyer" },
    "token": "jwt_token",
    "refreshToken": "jwt_refresh_token"
  }
}
```

Effets secondaires :

- Envoie un email de vérification
- Crée un panier vide pour l'utilisateur

Erreurs :

- 400: "Un compte existe déjà avec cet email"

4.3 POST /api/auth/logout

Description : Déconnexion (invalidation refresh token)

Auth : Oui (Bearer token)

Réponse 200 :

```
{ "message": "Déconnexion réussie" }
```

Effet : Supprime le refresh token de la BDD.

4.4 GET /api/auth/me

Description : Profil utilisateur courant

Auth : Oui

Réponse 200 :

```
{
  "data": {
    "id": "uuid",
    "email": "string",
    "name": "string",
    "phone": "string | null",
    "avatar": "string | null",
    "role": "buyer | seller | admin",
    "emailVerified": true,
    "createdAt": "2026-01-15T10:00:00.000Z"
  }
}
```

4.5 POST /api/auth/forgot-password

Description : Demande de réinitialisation de mot de passe

Auth : Non

Body :

```
{ "email": "string" }
```

Réponse 200 :

```
{ "message": "Si cet email existe, un lien de réinitialisation vous a été
```

Note : Retourne toujours 200 pour éviter l'énumération d'emails.

Effet : Génère un token de reset (expiration : 1h), envoie un email.

4.6 POST /api/auth/reset-password

Description : Réinitialisation du mot de passe

Auth : Non

Body :

```
{  
  "token": "string (token reçu par email)",  
  "password": "string (nouveau mot de passe)"  
}
```

Réponse 200 :

```
{ "message": "Mot de passe réinitialisé avec succès" }
```

Erreurs :

- 400: "Token invalide ou expiré"

4.7 POST /api/auth/verify-email

Description : Vérification de l'email

Auth : Non

Body :

```
{ "code": "string (code à 6 chiffres)" }
```

Réponse 200 :

```
{ "message": "Email vérifié avec succès" }
```

Erreurs :

- 400: "Code invalide ou expiré"

4.8 POST /api/auth/refresh-token

Description : Renouvellement du token d'accès

Auth : Non

Body :

```
{ "refreshToken": "string" }
```

Réponse 200 :

```
{  
  "data": {  
    "token": "nouveau_jwt_token",  
    "refreshToken": "nouveau_refresh_token"  
  }  
}
```

4.9 PUT /api/auth/profile

Description : Mise à jour du profil

Auth : Oui

Body :

```
{  
  "name": "string (optionnel)",  
  "phone": "string (optionnel)",  
  "avatar": "string (optionnel, URL)"  
}
```

Réponse 200 :

```
{ "data": { "id": " ... ", "name": " ... ", "phone": " ... ", "avatar": " ... " } }
```

4.10 PUT /api/auth/change-password

Description : Changement de mot de passe

Auth : Oui

Body :

```
{
  "currentPassword": "string",
  "newPassword": "string (min 8, 1 majuscule, 1 chiffre)"
}
```

Réponse 200 :

```
{ "message": "Mot de passe modifié avec succès" }
```

Erreurs :

- 400 : "Le mot de passe actuel est incorrect"

Chapitre 5 — API REST : Produits & Catégories

5.1 Produits

Base URL : /api/products

5.1.1 GET /api/products

Description : Liste paginée des produits

Auth : Non

Query params :

Param	Type	Défaut	Description
page	number	1	Numéro de page
limit	number	20	Éléments par page
sort	string	created_at	Champ de tri
order	string	desc	asc / desc
category	string	—	Slug de catégorie
minPrice	number	—	Prix minimum
maxPrice	number	—	Prix maximum
search	string	—	Recherche texte
status	string	active	Statut produit
seller	uuid	—	ID vendeur

Réponse 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "name": "string",
      "slug": "string",
      "price": 29.99,
      "compareAtPrice": 39.99,
      "images": [{ "url": "string", "alt": "string", "isPrimary": true }],
      "averageRating": 4.5,
      "reviewCount": 12,
      "stock": 50,
      "seller": { "id": "uuid", "shopName": "string", "rating": 4.8 },
      "category": { "id": "uuid", "name": "string", "slug": "string" },
      "status": "active",
      "createdAt": "2026-01-15T10:00:00.000Z"
    }
  ],
  "meta": { "page": 1, "limit": 20, "total": 150, "totalPages": 8 }
}
```

5.1.2 GET /api/products/:id

Description : Détail d'un produit

Auth : Non

Effet secondaire : Incrémente `viewCount`

Réponse 200 :

```
{
  "data": {
    "id": "uuid",
    "name": "string",
    "slug": "string",
    "description": "string (markdown)",
    "shortDescription": "string",
    "price": 29.99,
    "compareAtPrice": 39.99,
    "sku": "string",
    "stock": 50,
    "weight": 500,
    "dimensions": { "length": 20, "width": 15, "height": 10 },
    "images": [
      { "id": "uuid", "url": "string", "alt": "string", "isPrimary": true },
    ],
    "variants": [
      { "id": "uuid", "name": "Rouge / XL", "price": 29.99, "stock": 15, },
    ],
    "averageRating": 4.5,
    "reviewCount": 12,
    "salesCount": 150,
    "tags": ["smartphone", "apple"],
    "seller": {
      "id": "uuid",
      "shopName": "string",
      "slug": "string",
      "rating": 4.8,
      "totalSales": 500,
      "verified": true
    },
    "category": { "id": "uuid", "name": "Électronique", "slug": "electronique", "status": "active",
    "publishedAt": "2026-01-15T10:00:00.000Z",
    "createdAt": "2026-01-15T10:00:00.000Z"
  }
}
```

5.1.3 GET /api/products/slug/:slug

Description : Produit par slug (SEO-friendly)

Auth : Non

Réponse : Identique à 5.1.2

5.1.4 GET /api/products/category/:categorySlug

Description : Produits d'une catégorie

Auth : Non

Query : Mêmes params que 5.1.1

Réponse : Identique à 5.1.1

5.1.5 GET /api/products/search?q=...

Description : Recherche plein texte

Auth : Non

Query : q (requis) + params de pagination

Réponse : Identique à 5.1.1, avec results triés par pertinence (ts_rank)

5.1.6 GET /api/products/featured

Description : Produits en vedette

Auth : Non

Réponse : Tableau de produits (max 12)

5.1.7 GET /api/products/trending

Description : Produits tendance

Auth : Non

Réponse : Tableau de produits (max 12)

5.1.8 GET /api/products/new

Description : Nouveaux produits

Auth : Non

Réponse : Tableau de produits (max 12), triés par `publishedAt` DESC

5.1.9 GET /api/products/:id/related

Description : Produits similaires

Auth : Non

Réponse : Tableau de 4-8 produits de même catégorie

5.1.10 POST /api/products

Description : Créer un produit

Auth : Oui (seller)

Body :

```
{
  "name": "string",
  "categoryId": "uuid",
  "description": "string",
  "shortDescription": "string (optionnel)",
  "price": 29.99,
  "compareAtPrice": 39.99,
  "costPrice": 15.00,
  "sku": "string (optionnel)",
  "stock": 50,
  "lowStockThreshold": 5,
  "weight": 500,
  "dimensions": { "length": 20, "width": 15, "height": 10 },
  "tags": ["tag1", "tag2"],
  "variants": [
    { "name": "Rouge / XL", "price": 29.99, "stock": 15, "attributes": {

```

Réponse 201 : Produit créé

Effets : Génère le slug automatiquement

5.1.11 PUT /api/products/:id

Description : Modifier un produit

Auth : Oui (propriétaire du produit)

Body : Champs à modifier

Réponse 200 : Produit modifié

5.1.12 DELETE /api/products/:id

Description : Supprimer un produit

Auth : Oui (propriétaire)

Réponse 200 : { "message": "Produit supprimé" }

Effet : Supprime les images associées (physiquement et en BDD)

5.1.13 POST /api/products/:id/images

Description : Upload d'images

Auth : Oui (propriétaire)

Content-Type : multipart/form-data

Champ : images (multi-files, max 10)

Contraintes :

- Formats : jpg, jpeg, png, webp
- Taille max : 5 Mo par fichier
- Redimensionnement automatique : 800x800 (max)

Réponse 201 :

```
{
  "data": [
    { "id": "uuid", "url": "/uploads/products/abc123.webp", "alt": "", "is"
  ]
}
```

5.1.14 DELETE /api/products/:id/images/:imageId

Description : Supprimer une image

Auth : Oui (propriétaire)

Réponse 200 : { "message": "Image supprimée" }

Effet : Supprime le fichier physique

5.2 Catégories

Base URL : /api/categories

5.2.1 GET /api/categories

Description : Liste plate des catégories

Auth : Non

Réponse 200 : Tableau de catégories

```
{
  "data": [
    { "id": "uuid", "name": "Électronique", "slug": "electronique", "icon"
  ]
}
```

5.2.2 GET /api/categories/tree

Description : Arbre hiérarchique des catégories

Auth : Non

Réponse 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "name": "Électronique",
      "slug": "electronique",
      "icon": "📱",
      "children": [
        { "id": "uuid", "name": "Smartphones", "slug": "smartphones", "ico
```

5.2.3 GET /api/categories/:id

Description : Détail d'une catégorie

Auth : Non

Réponse 200 : Catégorie + infos produitCount

5.2.4 GET /api/categories/slug/:slug

Description : Catégorie par slug

Auth : Non

Réponse : Identique à 5.2.3

Chapitre 6 — API REST : Panier & Commandes

6.1 Panier

Base URL : /api/cart

6.1.1 GET /api/cart

Description : Récupérer le panier de l'utilisateur

Auth : Oui

Réponse 200 :

```
{
  "data": {
    "id": "uuid",
    "items": [
      {
        "id": "uuid",
        "product": {
          "id": "uuid",
          "name": "string",
          "price": 29.99,
          "image": "string",
          "stock": 50,
          "seller": { "id": "uuid", "shopName": "string" }
        },
        "variant": { "id": "uuid", "name": "Rouge / XL" },
        "quantity": 2
      }
    ],
    "coupon": null,
    "subtotal": 59.98,
    "total": 59.98,
    "totalItems": 2
  }
}
```

6.1.2 POST /api/cart/items

Description : Ajouter un produit au panier

Auth : Oui

Body :

```
{
  "productId": "uuid",
  "variantId": "uuid (optionnel)",
  "quantity": 1
}
```

Logique :

- Si le produit est déjà dans le panier → incrémente la quantité
 - Vérifie le stock disponible
 - Retourne le panier mis à jour
-

6.1.3 PUT /api/cart/items/:itemId

Description : Modifier la quantité

Auth : Oui

Body :

```
{ "quantity": 3 }
```

Logique :

- Si quantity = 0 → supprime l'élément
 - Vérifie le stock
-

6.1.4 DELETE /api/cart/items/:itemId

Description : Supprimer un élément du panier

Auth : Oui

Réponse 200 : Panier mis à jour

6.1.5 DELETE /api/cart

Description : Vider le panier

Auth : Oui

Réponse 200 : `{ "message": "Panier vidé" }`

6.1.6 POST /api/cart/coupon

Description : Appliquer un coupon

Auth : Oui

Body :

```
{ "code": "string" }
```

Réponse 200 :

```
{
  "data": {
    "coupon": { "code": "ETE2026", "type": "percentage", "value": 20 },
    "discount": 12.00,
    "total": 47.98
  }
}
```

Erreurs :

- 400: "Coupon invalide ou expiré"
- 400: "Montant minimum non atteint"

6.1.7 DELETE /api/cart/coupon

Description : Retirer le coupon

Auth : Oui

Réponse 200 : Panier mis à jour sans réduction

6.2 Commandes

Base URL : `/api/orders`

6.2.1 GET /api/orders

Description : Historique des commandes de l'utilisateur

Auth : Oui

Query : page , limit , status

Réponse 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "orderNumber": "GM-2026-000001",
      "status": "delivered",
      "totalAmount": 89.97,
      "currency": "EUR",
      "itemsCount": 3,
      "createdAt": "2026-06-15T10:00:00.000Z"
    }
  ],
  "meta": { ... }
}
```

6.2.2 GET /api/orders/:id

Description : Détail d'une commande

Auth : Oui (propriétaire ou admin)

Réponse 200 :


```
{
  "data": {
    "id": "uuid",
    "orderNumber": "GM-2026-000001",
    "status": "shipped",
    "subtotal": 79.98,
    "shippingCost": 5.99,
    "taxAmount": 0.00,
    "discountAmount": 10.00,
    "totalAmount": 75.97,
    "currency": "EUR",
    "notes": "Livrer avant 18h",
    "shippingMethod": "standard",
    "trackingNumber": "DHL123456789",
    "items": [
      {
        "id": "uuid",
        "productName": "iPhone 15 Pro",
        "productImage": "string",
        "quantity": 1,
        "unitPrice": 79.98,
        "totalPrice": 79.98,
        "seller": { "id": "uuid", "shopName": "TechStore" }
      }
    ],
    "address": {
      "firstName": "Jean",
      "lastName": "Dupont",
      "address1": "123 Rue de Paris",
      "city": "Paris",
      "postalCode": "75001",
      "country": "France"
    },
    "payments": [
      { "id": "uuid", "method": "credit_card", "amount": 75.97, "status": "paid" }
    ],
    "shipments": [
      { "id": "uuid", "carrier": "DHL", "trackingNumber": "DHL123 ...", "status": "in_transit" }
    ],
    "createdAt": "2026-06-15T10:00:00.000Z"
  }
}
```

6.2.3 POST /api/orders

Description : Créer une commande (checkout)

Auth : Oui

Body :

```
{
  "addressId": "uuid",
  "shippingMethod": "standard | express",
  "paymentMethod": "credit_card | paypal | mobile_money",
  "notes": "string (optionnel)",
  "couponCode": "string (optionnel)"
}
```

Logique de création :

1. Valider le panier (produits en stock, prix à jour)
2. Calculer les totaux (subtotal, shipping, tax, discount)
3. Générer `orderNumber` : `GM-YYYY-XXXXXX`
4. Créer la commande + `orderItems`
5. Décrémenter le stock des produits
6. Appliquer le commission vendeur
7. Vider le panier
8. Retourner la commande créée

Réponse 201 : Commande créée

6.2.4 POST /api/orders/:id/cancel

Description : Annuler une commande

Auth : Oui (propriétaire)

Condition : Seulement si status = `pending` ou `confirmed`

Logique :

- Remettre le stock
 - Marquer comme `cancelled`
 - Si déjà payée → déclencher un remboursement
-

6.2.5 GET /api/orders/:id/tracking

Description : Suivi de livraison

Auth : Oui (propriétaire)

Réponse 200 :

```
{
  "data": {
    "carrier": "DHL",
    "trackingNumber": "DHL123456789",
    "trackingUrl": "https://track.dhl.com/DHL123 ... ",
    "status": "in_transit",
    "estimatedDelivery": "2026-06-20T18:00:00.000Z",
    "events": [
      { "status": "picked_up", "location": "Paris", "timestamp": "2026-06-19T10:00:00.000Z" },
      { "status": "in_transit", "location": "Lyon", "timestamp": "2026-06-19T15:00:00.000Z" }
    ]
  }
}
```

6.2.6 GET /api/orders/:id/invoice

Description : Télécharger la facture PDF

Auth : Oui (propriétaire)

Content-Type : application/pdf

Content-Disposition : attachment; filename="facture-GM-2026-000001.pdf"

Contenu du PDF :

- En-tête GlobalMarket
- Numéro de commande
- Détail des articles
- Sous-total, frais, taxes, total
- Adresse de livraison
- Infos de paiement

Chapitre 7 — API REST : Paiements

Base URL : `/api/payments`

7.1 POST `/api/payments/intent`

Description : Créer une intention de paiement

Auth : Oui

Body :

```
{
  "orderId": "uuid",
  "amount": 75.97,
  "currency": "EUR",
  "method": "credit_card | paypal | mobile_money"
}
```

Logique selon la méthode :

Méthode	Action
credit_card	Créer un PaymentIntent Stripe → retourner <code>clientSecret</code>
paypal	Créer une commande PayPal → retourner <code>paypalOrderId</code> + <code>approvalUrl</code>
mobile_money	Initier le paiement → retourner <code>reference</code> + instructions

Réponse 200 :

```
{
  "data": {
    "paymentId": "uuid",
    "clientSecret": "pi_xxx_secret_xxx", // Stripe
    "paypalOrderId": "PAY-xxx", // PayPal
    "reference": "MOB-xxx" // Mobile Money
  }
}
```

7.2 POST /api/payments/:intentId/confirm

Description : Confirmer un paiement

Auth : Oui

Body (Stripe) :

```
{  
  "paymentMethodId": "pm_xxx"  
}
```

Body (PayPal) :

```
{  
  "payerId": "xxx",  
  "token": "EC-xxx"  
}
```

Logique :

1. Confirmer avec le provider (Stripe/PayPal)
2. Mettre à jour le statut Payment → **completed**
3. Mettre à jour la commande → **confirmed**
4. Envoyer email de confirmation

Réponse 200 :

```
{  
  "data": {  
    "status": "completed",  
    "transactionId": "pi_xxx"  
  }  
}
```

7.3 GET /api/payments/methods

Description : Méthodes de paiement disponibles

Auth : Non

Réponse 200 :

```
{
  "data": [
    { "id": "credit_card", "name": "Carte bancaire", "icon": "credit-card" },
    { "id": "paypal", "name": "PayPal", "icon": "paypal", "enabled": true },
    { "id": "apple_pay", "name": "Apple Pay", "icon": "apple", "enabled": true },
    { "id": "google_pay", "name": "Google Pay", "icon": "google", "enabled": true },
    { "id": "mobile_money", "name": "Mobile Money", "icon": "smartphone", "enabled": true },
  ]
}
```

7.4 POST /api/payments/:paymentId/refund

Description : Rembourser un paiement

Auth : Oui (admin)

Body :

```
{
  "amount": 75.97,
  "reason": "Produit défectueux"
}
```

Logique :

1. Créer le remboursement Stripe/PayPal
2. Mettre à jour Payment → `refunded` ou `partially_refunded`
3. Mettre à jour Order → `refunded`
4. Remettre le stock
5. Envoyer email de notification

7.5 GET /api/payments/history

Description : Historique des paiements (admin)

Auth : Oui (admin)

Query : `page`, `limit`, `status`, `method`, `dateFrom`, `dateTo`

Réponse 200 : Liste paginée de paiements

7.6 Webhooks Stripe

Route : POST /api/webhooks/stripe

Auth : Signature Stripe (pas JWT)

Événements gérés :

Événement	Action
payment_intent.succeeded	Confirmer le paiement
payment_intent.payment_failed	Marquer comme échoué
charge.refunded	Traiter le remboursement
charge.dispute.created	Notifier l'admin

7.7 Webhooks PayPal

Route : POST /api/webhooks/paypal

Auth : Signature PayPal

Événements gérés :

Événement	Action
PAYMENT.CAPTURE.COMPLETED	Confirmer le paiement
PAYMENT.CAPTURE.DENIED	Marquer comme échoué
PAYMENT.CAPTURE.REFUNDED	Traiter le remboursement

Chapitre 8 — API REST : Vendeur

Base URL : /api/seller

Auth requisite : Oui (rôle seller) sauf /register

8.1 POST /api/seller/register

Description : Inscription vendeur (un buyer peut devenir seller)

Auth : Oui (buyer)

Body :

```
{  
  "shopName": "Ma Boutique",  
  "description": "Description de ma boutique",  
  "category": "Électronique",  
  "country": "France",  
  "businessType": "individual | company"  
}
```

Logique :

1. Valider l'unicité du shopName
2. Créer le profil Seller
3. Générer le slug à partir du shopName
4. Mettre à jour le rôle User → `seller`

Réponse 201 : Profil vendeur créé

8.2 GET /api/seller/dashboard

Description : Tableau de bord vendeur

Auth : Oui (seller)

Réponse 200 :


```
{
  "data": {
    "stats": {
      "totalSales": 1250,
      "totalRevenue": 45678.50,
      "pendingOrders": 8,
      "activeProducts": 45,
      "averageRating": 4.7,
      "conversionRate": 3.2
    },
    "recentOrders": [
      { "id": "uuid", "orderNumber": "GM-2026-000123", "totalAmount": 89.9
    ],
    "salesChart": [
      { "date": "2026-07-01", "sales": 1200, "orders": 15 },
      { "date": "2026-07-02", "sales": 980, "orders": 12 }
    ],
    "topProducts": [
      { "id": "uuid", "name": "iPhone 15", "salesCount": 45, "revenue": 5
    ]
  }
}
```

8.3 GET /api/seller/shop

Description : Profil de la boutique

Auth : Oui (seller)

Réponse 200 : Détail du profil seller + paramètres

8.4 PUT /api/seller/shop

Description : Modifier la boutique

Auth : Oui (seller)

Body : Champs modifiables (shopName, description, logo, banner, etc.)

Réponse 200 : Profil mis à jour

8.5 GET /api/seller/products

Description : Produits du vendeur

Auth : Oui (seller)

Query : `page` , `limit` , `status` , `search` , `sort`

Réponse 200 : Liste paginée des produits du vendeur (tous statuts)

8.6 POST /api/seller/products

Description : Créer un produit

Auth : Oui (seller)

Body : Identique à 5.1.10 (POST /api/products)

Effet : `sellerId` est automatiquement `req.user.sellerId`

8.7 PUT /api/seller/products/:id

Description : Modifier un produit du vendeur

Auth : Oui (propriétaire)

Vérification : Le produit appartient au vendeur courant

8.8 DELETE /api/seller/products/:id

Description : Supprimer un produit

Auth : Oui (propriétaire)

Condition : Pas de commandes en cours pour ce produit

8.9 GET /api/seller/orders

Description : Commandes contenant des produits du vendeur

Auth : Oui (seller)

Query : page , limit , status , dateFrom , dateTo

Réponse 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "orderNumber": "GM-2026-000123",
      "buyer": { "name": "Jean Dupont" },
      "items": [
        { "productName": "iPhone 15", "quantity": 1, "unitPrice": 79.98 }
      ],
      "totalAmount": 79.98,
      "status": "confirmed",
      "createdAt": " ..."
    }
  ],
  "meta": { ... }
}
```

8.10 GET /api/seller/orders/:id

Description : Détail d'une commande vendeur

Auth : Oui (seller)

Vérification : Le vendeur a au moins un item dans cette commande

8.11 PATCH /api/seller/orders/:id/status

Description : Mettre à jour le statut d'un item de commande

Auth : Oui (seller)

Body :

```
{ "status": "processing | shipped | delivered" }
```

Logique :

- Si shipped → enregistrer le numéro de suivi

- Si tous les items d'une commande sont livrés → commande = livrée

8.12 GET /api/seller/analytics

Description : Statistiques avancées du vendeur

Auth : Oui (seller)

Query : `period` (7d | 30d | 90d | 1y)

Réponse 200 :

```
{
  "data": {
    "revenue": { "total": 45678.50, "change": 12.5 },
    "orders": { "total": 350, "change": 8.2 },
    "averageOrderValue": 130.51,
    "conversionRate": 3.2,
    "topProducts": [ ... ],
    "salesByDay": [ ... ],
    "salesByCategory": [ ... ],
    "customerRetention": 45.2
  }
}
```

8.13 GET /api/seller/payouts

Description : Historique des virements

Auth : Oui (seller)

Réponse 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "amount": 1250.00,
      "currency": "EUR",
      "status": "completed",
      "method": "bank_transfer",
      "paidAt": "2026-07-01T10:00:00Z"
    }
  ]
}
```

8.14 PUT /api/seller/bank-account

Description : Mettre à jour les infos bancaires

Auth : Oui (seller)

Body :

```
{
  "iban": "FR76 ... ",
  "bic": "BNPAFRPP",
  "holderName": "Jean Dupont"
}
```

Chapitre 9 — API REST :

Administration

Base URL : /api/admin

Auth require : Oui (rôle admin)

9.1 GET /api/admin/dashboard

Description : Tableau de bord administrateur

Réponse 200 :

```
{
  "data": {
    "stats": {
      "totalUsers": 125000,
      "totalSellers": 8500,
      "totalProducts": 450000,
      "totalOrders": 89000,
      "totalRevenue": 2500000,
      "monthlyRevenue": 450000,
      "pendingOrders": 120,
      "pendingPayouts": 45,
      "openTickets": 28
    },
    "recentOrders": [ ... ],
    "newUsers": [ ... ],
    "revenueChart": [ ... ],
    "topSellers": [ ... ],
    "alerts": [
      { "type": "warning", "message": "15 produits en rupture de stock" },
      { "type": "info", "message": "12 nouvelles demandes vendeur" }
    ]
  }
}
```

9.2 GET /api/admin/users

Description : Liste des utilisateurs

Query : `page`, `limit`, `role`, `status`, `search`, `sort`

Réponse 200 : Liste paginée

```
{
  "data": [
    {
      "id": "uuid",
      "name": "Jean Dupont",
      "email": "jean@email.com",
      "role": "buyer",
      "status": "active",
      "ordersCount": 15,
      "totalSpent": 1250.00,
      "createdAt": " ..."
    }
  ]
}
```

9.3 GET /api/admin/users/:id

Description : Détail d'un utilisateur

Réponse 200 : Profil complet + adresses + dernières commandes

9.4 PATCH /api/admin/users/:id/status

Description : Changer le statut d'un utilisateur

Body :

```
{ "status": "active | inactive | suspended | banned" }
```

Effet : Si banned → supprime tous les refresh tokens

9.5 PATCH /api/admin/users/:id/role

Description : Changer le rôle d'un utilisateur

Body :

```
{ "role": "buyer | seller | admin | delivery" }
```

9.6 GET /api/admin/products

Description : Tous les produits (modération)

Query : `page`, `limit`, `status`, `category`, `search`, `sort`

Réponse 200 : Liste paginée avec infos vendeur

9.7 PATCH /api/admin/products/:id/moderate

Description : Modérer un produit

Body :

```
{ "action": "approve | reject | suspend" }
```

Effets :

- approve → status = `active`
 - reject → status = `inactive` + notification au vendeur
 - suspend → status = `suspended` + notification
-

9.8 GET /api/admin/orders

Description : Toutes les commandes

Query : `page`, `limit`, `status`, `dateFrom`, `dateTo`, `seller`, `search`

Réponse 200 : Liste paginée

9.9 PATCH /api/admin/orders/:id/status

Description : Modifier le statut d'une commande (admin override)

Body :

```
{ "status": "processing | shipped | delivered | cancelled | refunded" }
```

9.10 POST /api/admin/orders/:id/refund

Description : Rembourser une commande (admin)

Body :

```
{  
  "amount": 75.97,  
  "reason": "Litige résolu en faveur de l'acheteur"  
}
```

Effets :

1. Créer le remboursement (Stripe/PayPal)
2. Mettre à jour paiement + commande
3. Remettre le stock
4. Notifier l'acheteur et le vendeur

9.11 GET /api/admin/reports/:type

Description : Rapports et statistiques

Types disponibles :

Type	Description
sales	Ventes par période
revenue	Revenus par catégorie/vendeur
users	Inscriptions, activité

Type	Description
products	Produits populaires, catégories
sellers	Performance vendeurs
payouts	Virements effectués

Query : `period` (7d | 30d | 90d | 1y), `groupBy` (day | week | month)

9.12 GET /api/admin/settings

Description : Paramètres système

Réponse 200 :

```
{
  "data": {
    "siteName": "GlobalMarket",
    "siteDescription": "Marketplace mondiale",
    "commissionRate": 0.12,
    "paymentMethods": ["credit_card", "paypal", "mobile_money"],
    "defaultShippingRate": 5.99,
    "shippingFreeThreshold": 50,
    "emailOrderConfirmation": true,
    "emailOrderShipped": true,
    "emailNewSeller": true,
    "emailLowStock": true
  }
}
```

9.13 PUT /api/admin/settings

Description : Modifier les paramètres système

Body : Champs modifiables

Effet : Invalider le cache Redis

Chapitre 10 — API REST : Support & Uploads

10.1 Support

Base URL : `/api/support`

10.1.1 GET `/api/support/tickets`

Description : Liste des tickets de support

Auth : Oui

Règle :

- Buyer/Admin voient tous les tickets
- Seller voit les siens + tickets liés à ses produits
- Query : `page` , `limit` , `status` , `priority` , `category`

Réponse 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "subject": "Commande non reçue",
      "category": "order",
      "priority": "high",
      "status": "open",
      "messagesCount": 3,
      "lastMessage": { "message": " ... ", "createdAt": " ... " },
      "createdAt": " ... "
    }
  ]
}
```

10.1.2 POST /api/support/tickets

Description : Créer un ticket

Auth : Oui

Body :

```
{
  "subject": "Commande non reçue",
  "category": "order | product | account | technical | other",
  "priority": "low | medium | high | urgent",
  "orderId": "uuid (optionnel)",
  "message": "Description du problème"
}
```

Réponse 201 : Ticket créé

10.1.3 GET /api/support/tickets/:id

Description : Détail d'un ticket + messages

Auth : Oui (propriétaire ou admin)

Réponse 200 :

```
{
  "data": {
    "id": "uuid",
    "subject": "Commande non reçue",
    "category": "order",
    "priority": "high",
    "status": "in_progress",
    "order": { "id": "uuid", "orderNumber": "GM-2026-000123" },
    "messages": [
      {
        "id": "uuid",
        "sender": { "id": "uuid", "name": "Jean", "role": "buyer" },
        "message": "Ma commande du 15 juin n'est pas arrivée",
        "isInternal": false,
        "createdAt": " ... "
      }
    ],
    "createdAt": " ... "
  }
}
```

10.1.4 POST /api/support/tickets/:ticketId/messages

Description : Ajouter un message

Auth : Oui (propriétaire ou admin)

Body :

```
{  
  "message": "Nous avons contacté le transporteur",  
  "attachments": ["url1", "url2"]  
}
```

Effets :

- Incrémente le compteur de messages
 - Met à jour `updatedAt`
 - Si admin → notifie l'utilisateur
 - Si utilisateur → notifie l'admin
-

10.1.5 POST /api/support/tickets/:id/close

Description : Fermer un ticket

Auth : Oui (propriétaire ou admin)

Réponse 200 :

```
{ "message": "Ticket fermé" }
```

10.1.6 GET /api/support/kb

Description : Base de connaissances

Auth : Non

Query : `search` , `category`

Réponse 200 :

```
{
  "data": [
    {
      "id": "uuid",
      "title": "Comment suivre ma commande",
      "category": "Commandes",
      "excerpt": "...",
      "slug": "suivre-commande"
    }
  ]
}
```

10.1.7 GET /api/support/kb/:id

Description : Article de la base de connaissances

Auth : Non

Réponse 200 : Article complet (markdown)

10.2 Uploads

Base URL : /api/uploads

10.2.1 POST /api/uploads/image

Description : Upload d'une image

Auth : Oui

Content-Type : multipart/form-data

Champ : file

Contraintes :

- Formats : jpg, jpeg, png, webp, gif
- Taille max : 5 Mo

Réponse 201 :

```
{
  "data": {
    "id": "uuid",
    "url": "/uploads/images/abc123.webp",
    "filename": "abc123.webp",
    "mimetype": "image/webp",
    "size": 245000
  }
}
```

10.2.2 POST /api/uploads/images

Description : Upload multiple

Auth : Oui

Champ : `files` (max 10)

Réponse 201 : Tableau d'images uploadées

10.2.3 POST /api/uploads/document

Description : Upload de document

Auth : Oui

Champ : `file`

Formats autorisés : pdf, doc, docx

Taille max : 10 Mo

10.2.4 DELETE /api/uploads/:fileId

Description : Supprimer un fichier

Auth : Oui (propriétaire)

Effet : Supprime le fichier physique + l'enregistrement BDD

Chapitre 11 — Sécurité & Authentification

11.1 Authentification JWT

11.1.1 Tokens

Token	Durée	Stockage	Usage
Access Token	7 jours	Header <code>Authorization</code>	Authentifier chaque requête
Refresh Token	30 jours	HTTP-only cookie + BDD	Renouveler l'access token

11.1.2 Structure du JWT

```
{
  "sub": "uuid utilisateur",
  "email": "string",
  "role": "buyer | seller | admin | delivery",
  "iat": 1753000000,
  "exp": 1753604800
}
```

11.1.3 Flux d'authentification

1. Login → credentials validés → JWT signé → retourné au client
2. Client stocke le token (localStorage)
3. Chaque requête : `Authorization: Bearer <token>`
4. Middleware `auth.js` : vérifie la signature, injecte `req.user`
5. Token expiré → 401 → client appelle `/refresh-token`
6. Refresh token invalide → 401 → reconnexion requise

11.2 Hachage des mots de passe

Algorithme : bcrypt (12 rounds)


```
// Hachage
const hash = await bcrypt.hash(password, 12)

// Vérification
const valid = await bcrypt.compare(inputPassword, hash)
```

11.3 RBAC (Role-Based Access Control)

```
// Middleware de rôle
function requireRole(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ message: 'Accès interdit' })
    }
    next()
  }
}

// Utilisation
router.get('/admin/users', auth, requireRole('admin'), adminController.get)
router.post('/seller/products', auth, requireRole('seller'), sellerContro
```

Matrice d'accès

Ressource	Buyer	Seller	Admin	Delivery
Lire produits	✓	✓	✓	✓
Créer produit	✗	✓	✓	✗
Modifier produit	✗	Propriétaire	✓	✗
Lire commandes	Propriétaire	Liées	✓	Livraisons
Créer commande	✓	✗	✓	✗
Gérer paiements	✗	✗	✓	✗
Gérer utilisateurs	✗	✗	✓	✗
Stats vendeur	✗	Propriétaire	✓	✗
Tickets support	Propriétaire	Propriétaire	✓	✗

11.4 Validation des entrées

Outil : Zod

```
const loginSchema = z.object({
  email: z.string().email('Email invalide'),
  password: z.string().min(8, 'Le mot de passe doit contenir au moins 8 caractères'),
})

// Middleware
function validate(schema) {
  return (req, res, next) => {
    const result = schema.safeParse(req.body)
    if (!result.success) {
      return res.status(400).json({ message: result.error.issues[0].message })
    }
    req.body = result.data
    next()
  }
}
```

11.5 Protection contre les attaques

Attaque	Protection
Brute force	Rate limiting (100 req/min par IP)
SQL injection	Prisma ORM (pas de SQL brut)
XSS	Sanitization des entrées, Content-Security-Policy
CSRF	Token SameSite cookie + Origin header
Man-in-the-middle	HTTPS obligatoire en production
Enumeration d'emails	Réponses uniformisées (toujours 200 pour forgot-password)
Fichier volumineux	Limitation taille upload (5 Mo image, 10 Mo document)
Injections NoSQL	Validation Zod sur tous les inputs

11.6 Rate Limiting

```
// Par IP et par route
const apiLimiter = rateLimit({
  windowMs: 60 * 1000,      // 1 minute
  max: 100,                 // 100 requêtes par minute
  message: { message: 'Trop de requêtes, veuillez réessayer' }
})

// Routes sensibles (login, register)
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 10,                  // 10 tentatives
  message: { message: 'Trop de tentatives, veuillez réessayer plus tard' }
})
```

11.7 Headers de sécurité (Helmet)

```
const helmet = require('helmet')

app.use(helmet({
  contentSecurityPolicy: true,
  crossOriginEmbedderPolicy: false,
  crossOriginResourcePolicy: { policy: "cross-origin" },
  crossOriginOpenerPolicy: false
}))
```

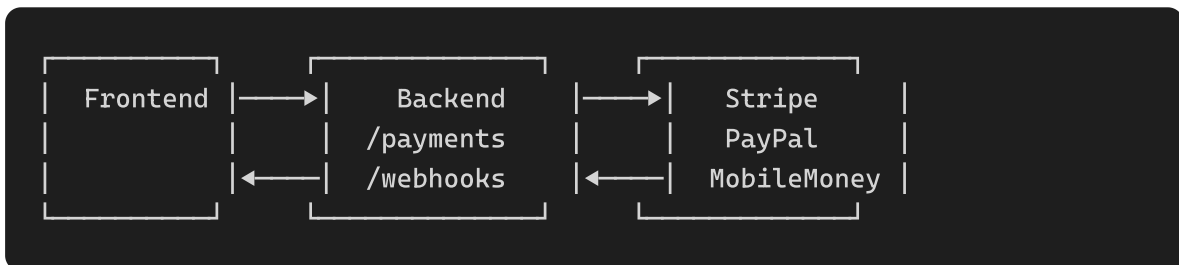
11.8 Conformité RGPD

Exigence	Implémentation
Droit d'accès	GET /api/auth/me + export données
Droit de rectification	PUT /api/auth/profile
Droit à l'effacement	DELETE /api/auth/account (soft delete)
Droit à la portabilité	Export JSON des données utilisateur
Consentement cookies	Bannière de consentement frontend

Exigence	Implémentation
Minimisation données	Seules les données nécessaires sont collectées
Chiffrement	Bcrypt (MDP), HTTPS (transport), AES-256 (données sensibles)

Chapitre 12 — Intégrations Paiement

12.1 Architecture des paiements



12.2 Stripe

12.2.1 Configuration

```
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY)
```

12.2.2 Flux carte bancaire

1. Frontend → POST /payments/intent { amount, currency, orderId }
2. Backend → stripe.paymentIntents.create({ amount: 7597, currency: 'eur' }
3. Backend → retourne { clientSecret: 'pi_xxx_secret_xxx' }
4. Frontend → stripe.confirmCardPayment(clientSecret, { payment_method: {
5. Stripe → webhook payment_intent.succeeded
6. Backend → met à jour Payment + Order

12.2.3 Signature webhook

```
app.post('/api/webhooks/stripe', express.raw({ type: 'application/json' }));
const sig = req.headers['stripe-signature'];
const event = stripe.webhooks.constructEvent(req.body, sig, process.env.STRIPE_SECRET_KEY);
// ... traiter l'événement
res.json({ received: true });
})
```

12.3 PayPal

12.3.1 Configuration

```
const paypal = require('@paypal/checkout-server-sdk');
const client = new paypal.core.PayPalHttpClient(
  new paypal.core.SandboxEnvironment(process.env.PAYPAL_CLIENT_ID, process.env.PAYPAL_SECRET_KEY)
);
```

12.3.2 Flux PayPal

1. Frontend → POST /payments/intent { amount, currency, orderId, method: 'paypal' }
2. Backend → créer l'ordre PayPal
3. Backend → retourne { paypalOrderId, approvalUrl }
4. Frontend → redirige vers approvalUrl
5. User approuve → retourne vers frontend avec token
6. Frontend → POST /payments/:id/confirm { payerId, token }
7. Backend → capture le paiement
8. PayPal → webhook PAYMENT.CAPTURE.COMPLETED

12.4 Mobile Money (Orange Money, M-Pesa, Wave)

12.4.1 Architecture

1. Frontend → POST /payments/intent { amount, currency, orderId, method:
2. Backend → initie le paiement via l'API du provider
3. Backend → envoie une demande de paiement au téléphone
4. User confirme sur son téléphone
5. Provider → webhook de confirmation
6. Backend → met à jour le statut

12.4.2 Providers supportés

Provider	Pays	API
Orange Money	Sénégal, Côte d'Ivoire, Mali	REST API
M-Pesa	Kenya, Tanzanie	Daraja API
Wave	Sénégal, Côte d'Ivoire	REST API
MTN Mobile Money	Cameroun, Burkina Faso	REST API

12.5 Système d'escrow

Le paiement n'est pas transféré directement au vendeur. Le flux :

1. Acheteur paie → fonds détenus par GlobalMarket (escrow)
2. Produit livré + confirmation → délai de réclamation (48h)
3. Pas de litige → fonds transférés au vendeur (moins commission)
4. Litige → fonds gelés jusqu'à résolution

Calcul commission

```
Montant vendeur = totalArticle × (1 - commissionRate)
Commission = totalArticle × commissionRate
```

Exemple :

Produit : 100 €

Commission Basic : 12% → Vendeur reçoit 88 €

Commission Pro : 8% → Vendeur reçoit 92 €

Commission Enterprise : 5% → Vendeur reçoit 95 €

12.6 Payouts (Virements vendeurs)

Fréquence

Plan	Fréquence minimum	Montant minimum
Basic	Mensuel	50 €
Pro	Hebdomadaire	25 €
Enterprise	Hebdomadaire	10 €

Processus

1. Admin lance le batch de payouts (cron job)
2. Pour chaque vendeur éligible :
 - a. Calculer le solde disponible (revenus - commissions - remboursements)
 - b. Créer un enregistrement Payout
 - c. Initier le virement bancaire / PayPal
 - d. Mettre à jour le statut
3. Notifier le vendeur par email

Chapitre 13 — Notifications & Emails

13.1 Système de notifications

13.1.1 Types de notifications

Type	Canal	Déclencheur
Commande confirmée	Email + DB	Paielement réussi
Commande expédiée	Email + DB + Push	Seller change statut → shipped
Commande livrée	Email + DB	Confirmation livraison
Commande annulée	Email + DB	Annulation par buyer ou seller
Remboursement	Email + DB	Remboursement traité
Nouveau produit vendeur	DB	Création produit
Produit modéré	Email + DB	Approbation/rejet admin
Nouveau ticket	DB	Création ticket support
Réponse ticket	Email + DB	Nouveau message admin
Compte vendeur approuvé	Email + DB	Verification admin
Solde bas	Email	Stock < lowStockThreshold
Nouvel abonné newsletter	Email	Inscription newsletter

13.1.2 Modèle de notification (DB)

```
{
  "id": "uuid",
  "userId": "uuid",
  "type": "order | payment | shipping | promotion | system | support",
  "title": "Commande confirmée",
  "message": "Votre commande GM-2026-000123 a été confirmée",
  "data": { "orderId": "uuid", "orderNumber": "GM-2026-000123" },
  "read": false,
  "createdAt": "2026-07-23T10:00:00Z"
}
```


13.2 Emails transactionnels

13.2.1 Templates

Template	Sujet	Contenu
verify-email	Vérifiez votre email	Code à 6 chiffres
reset-password	Réinitialisez votre mot de passe	Lien de réinitialisation
order-confirmation	Confirmation de commande #{number}	Détails commande
order-shipped	Votre commande #{number} est en route	Tracking
order-delivered	Votre commande #{number} a été livrée	Récapitulatif
order-cancelled	Annulation de commande #{number}	Raison
refund-processed	Remboursement traité	Montant et délai
seller-approved	Votre boutique est approuvée	Bienvenue
seller-rejected	Votre demande de boutique rejetée	Raison
new-ticket	Nouveau ticket #{id}	Résumé
ticket-reply	Réponse à votre ticket	Message
low-stock	Alerte stock bas	Produit concerné
payout-completed	Virement effectué	Montant

13.2.2 Service d'envoi

```
// email.service.js
const nodemailer = require('nodemailer')

const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST,
  port: process.env.SMTP_PORT,
  secure: true,
  auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS }
})

async function sendEmail(to, subject, html) {
  await transporter.sendMail({
    from: process.env.EMAIL_FROM,
    to,
    subject,
    html
  })
}
```

13.2.3 Intégration transactionnelle

```
// Dans le service order
async function createOrder(data) {
  return await prisma.$transaction(async (tx) => {
    // 1. Créer la commande
    const order = await tx.order.create({ ... })

    // 2. Décrémenter le stock
    for (const item of items) {
      await tx.product.update({
        where: { id: item.productId },
        data: { stock: { decrement: item.quantity } }
      })
    }

    // 3. Créer la notification
    await tx.notification.create({
      data: { userId: buyerId, type: 'order', title: 'Commande confirmée' }
    })

    // 4. Envoyer l'email (hors transaction)
    await emailService.sendOrderConfirmation(order)

    return order
  })
}
```

13.3 Notifications push (futur)

```
// Service push via Firebase Cloud Messaging
async function sendPushNotification(userId, title, body, data) {
  const tokens = await getUserPushTokens(userId)
  await admin.messaging().sendEachForMulticast({
    tokens,
    notification: { title, body },
    data
  })
}
```

Chapitre 14 — Déploiement & DevOps

14.1 Docker

14.1.1 Dockerfile (Backend)

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/prisma ./prisma
COPY --from=builder /app/package.json ./

ENV NODE_ENV=production
EXPOSE 5000

CMD ["sh", "-c", "npx prisma migrate deploy && node dist/app.js"]
```

14.1.2 docker-compose.yml

```
version: '3.8'

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: globalmarket
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: globalmarket_db
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U globalmarket"]
      interval: 5s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redisdata:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s

  backend:
    build:
      context: .
      dockerfile: docker/Dockerfile
    ports:
      - "5000:5000"
    environment:
      DATABASE_URL: postgresql://globalmarket:secret@postgres:5432/globalmarket
      REDIS_URL: redis://redis:6379
      NODE_ENV: production
      JWT_SECRET: ${JWT_SECRET}
      STRIPE_SECRET_KEY: ${STRIPE_SECRET_KEY}
    depends_on:
      postgres:
        condition: service_healthy
      redis:
```

```
    condition: service_healthy
  volumes:
    - uploads:/app/uploads

nginx:
  image: nginx:alpine
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf
    - ./nginx/certs:/etc/nginx/certs
  depends_on:
    - backend

volumes:
  pgdata:
  redisdata:
  uploads:
```

14.2 CI/CD

14.2.1 Pipeline GitHub Actions

```
name: Backend CI/CD

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
          POSTGRES_DB: test_db
        ports: ["5432:5432"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx prisma migrate deploy
      env:
        DATABASE_URL: postgresql://test:test@localhost:5432/test_db
      - run: npm run test
      - run: npm run lint

  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to production
        run: |
          # Deploy via Docker Compose or cloud provider
          docker compose -f docker-compose.prod.yml up -d --build
```

14.3 Variables d'environnement (production)

Variable	Description	Sensibilité
DATABASE_URL	URL PostgreSQL	🔒 Secret
JWT_SECRET	Clé de signature JWT	🔒 Secret
JWT_REFRESH_SECRET	Clé refresh token	🔒 Secret
STRIPE_SECRET_KEY	Clé API Stripe	🔒 Secret
STRIPE_WEBHOOK_SECRET	Secret webhook Stripe	🔒 Secret
PAYPAL_CLIENT_ID	ID client PayPal	🔒 Secret
PAYPAL_CLIENT_SECRET	Secret PayPal	🔒 Secret
SMTP_PASS	Mot de passe SMTP	🔒 Secret
REDIS_URL	URL Redis	⚠️ Interne
NODE_ENV	Environnement	📘 Public
PORT	Port serveur	⚙️ Config

14.4 Monitoring & Logs

14.4.1 Health check

GET /api/health

```
{
  "status": "ok",
  "uptime": 86400,
  "database": "connected",
  "redis": "connected",
  "version": "1.0.0"
}
```


14.4.2 Logs structurés

```
{
  "timestamp": "2026-07-23T10:30:00.000Z",
  "level": "info",
  "message": "Order created",
  "orderId": "uuid",
  "userId": "uuid",
  "amount": 89.97,
  "requestId": "req_xxx",
  "ip": "192.168.1.1"
}
```

14.4.3 Métriques clés

Métrique	Seuil d'alerte
Temps de réponse API (p95)	> 500ms
Taux d'erreur 5xx	> 1%
CPU utilisation	> 80%
RAM utilisation	> 85%
Connexions DB actives	> 80
Queue Redis	> 1000

14.5 Stratégie de backup

Composant	Fréquence	Rétention	Méthode
PostgreSQL	Quotidien	30 jours	pg_dump + S3
PostgreSQL WAL	Continue	7 jours	Streaming replication
Redis	Quotidien	7 jours	RDB snapshot
Uploads	Continu	—	S3 sync

14.6 Scaling

Phase 1 (0-10K users)

- 1 serveur backend (2 vCPU, 4 GB RAM)
- 1 PostgreSQL (managed)
- 1 Redis (managed)

Phase 2 (10K-100K users)

- 2+ serveurs backend (load balancer)
- PostgreSQL avec read replicas
- Redis Cluster

Phase 3 (100K+ users)

- Kubernetes (EKS/GKE)
- PostgreSQL avec connection pooling (PgBouncer)
- CDN pour les uploads (CloudFront)
- Microservices (payment, notification)

Chapitre 15 — Tests & Qualité

15.1 Stratégie de tests

Type	Couverture cible	Outils
Unit tests	Services, utils, validators	Vitest
Integration tests	API endpoints	Vitest + Supertest
E2E tests	Parcours critiques	Playwright (futur)

15.2 Exemples de tests

Test unitaire (service)

```
// tests/unit/auth.service.test.js
import { describe, it, expect, vi } from 'vitest'
import { hashPassword, verifyPassword } from '../src/utils/hash'

describe('hashPassword', () => {
  it('should hash a password', async () => {
    const hash = await hashPassword('MonMotDePasse123')
    expect(hash).not.toBe('MonMotDePasse123')
    expect(hash.length).toBeGreaterThan(50)
  })
})

describe('verifyPassword', () => {
  it('should verify correct password', async () => {
    const hash = await hashPassword('test123')
    expect(await verifyPassword('test123', hash)).toBe(true)
    expect(await verifyPassword('wrong', hash)).toBe(false)
  })
})
```

Test d'intégration (API)

```
// tests/integration/auth.test.js
import { describe, it, expect, beforeAll, afterAll } from 'vitest'
import request from 'supertest'
import { app } from '../../src/app'

describe('POST /api/auth/register', () => {
  it('should register a new user', async () => {
    const res = await request(app)
      .post('/api/auth/register')
      .send({
        name: 'Jean Test',
        email: 'jean@test.com',
        password: 'SecurePass123'
      })

    expect(res.status).toBe(201)
    expect(res.body.data.user.email).toBe('jean@test.com')
    expect(res.body.data.token).toBeDefined()
  })

  it('should reject duplicate email', async () => {
    const res = await request(app)
      .post('/api/auth/register')
      .send({
        name: 'Jean Test',
        email: 'jean@test.com',
        password: 'SecurePass123'
      })

    expect(res.status).toBe(400)
    expect(res.body.message).toContain('existe déjà')
  })
})
```

Test de commande (flux complet)

```
// tests/integration/order.test.js
describe('POST /api/orders', () => {
  it('should create an order from cart', async () => {
    // 1. Ajouter au panier
    await request(app)
      .post('/api/cart/items')
      .set('Authorization', `Bearer ${buyerToken}`)
      .send({ productId: product.id, quantity: 2 })

    // 2. Créer la commande
    const res = await request(app)
      .post('/api/orders')
      .set('Authorization', `Bearer ${buyerToken}`)
      .send({
        addressId: address.id,
        shippingMethod: 'standard',
        paymentMethod: 'credit_card'
      })

    expect(res.status).toBe(201)
    expect(res.body.data.orderNumber).toMatch(/^GM-\d{4}-\d{6}$/)
    expect(res.body.data.status).toBe('pending')

    // 3. Vérifier que le stock a diminué
    const productAfter = await prisma.product.findUnique({ where: { id: product.id } })
    expect(productAfter.stock).toBe(48) // 50 - 2

  })
})
```

15.3 Linting & Formatting

Outil	Configuration
ESLint	Règles strictes, no-any, unused-vars
Prettier	Formatage automatique
Husky	Pre-commit hooks
lint-staged	Lint only staged files

```
// package.json
{
  "scripts": {
    "lint": "eslint src/ --ext .js",
    "lint:fix": "eslint src/ --ext .js --fix",
    "format": "prettier --write 'src/**/*.js'",
    "test": "vitest",
    "test:coverage": "vitest --coverage",
    "test:ui": "vitest --ui"
  }
}
```

15.4 CI Quality Gates

Gate	Seuil
Tests passent	100%
Couverture minimale	70%
Lint erreurs	0
Build réussi	✓
Pas de dépendances vulnérables	0 high/critical

Chapitre 16 — Roadmap & Planning

16.1 Phase 1 — Fondation (Semaines 1-4)

Semaine	Tâches	Livrables
S1	Setup projet, Docker, Prisma, DB schema	Repo, DB opérationnelle
S2	Auth (register, login, JWT, RBAC)	API auth complète
S3	Produits CRUD, catégories, upload images	API produits
S4	Panier, coupons, commandes	API panier + commandes

16.2 Phase 2 — Paiements & Vendeur (Semaines 5-8)

Semaine	Tâches	Livrables
S5	Intégration Stripe, webhooks	Paiement carte
S6	PayPal, Mobile Money	Multi-paiement
S7	Dashboard vendeur, produits vendeur	Espace vendeur
S8	Analytics vendeur, payouts	Vendeur complet

16.3 Phase 3 — Admin & Support (Semaines 9-11)

Semaine	Tâches	Livrables
S9	Dashboard admin, gestion users	Admin users
S9	Modération produits, gestion commandes	Admin orders
S10	Rapports, paramètres système	Admin complet
S11	Tickets support, base de connaissances	Support

16.4 Phase 4 — Notifications & Optimisation (Semaines 12-14)

Semaine	Tâches	Livrables
S12	Emails transactionnels, templates	Notifications email
S12	Notifications push, in-app	Push notifications
S13	Performance (caching, indexes, N+1)	Optimisation
S13	Tests (unit, integration, coverage)	Qualité
S14	Sécurité (audit OWASP, pentest)	Sécurité
S14	Documentation API (Swagger)	Docs

16.5 Phase 5 — Déploiement (Semaines 15-16)

Semaine	Tâches	Livrables
S15	CI/CD, Docker prod, monitoring	Pipeline
S15	Backup, restore, runbooks	Ops
S16	Staging test, load test	Validation
S16	Production deploy	Mise en ligne

16.6 Estimation des coûts (infrastructure)

Phase 1 (Development)

Ressource	Coût mensuel
VPS (2 vCPU, 4 GB)	~20 €
PostgreSQL managed	~25 €
Redis managed	~15 €
Domaine + SSL	~10 €
Total	~70 €/mois

Phase 2 (Production - 10K users)

Ressource	Coût mensuel
App server (4 vCPU, 8 GB)	~60 €
PostgreSQL (4 vCPU, 16 GB)	~80 €
Redis (2 GB)	~30 €
CDN + Storage	~20 €
Email (SendGrid)	~20 €
Monitoring (Datadog)	~25 €
Total	~235 €/mois

Phase 3 (Scale - 100K+ users)

Ressource	Coût mensuel
Kubernetes (3 nodes)	~300 €
PostgreSQL HA	~200 €
Redis Cluster	~100 €
CDN + Storage	~100 €
Email + Push	~50 €
Monitoring + Logs	~100 €
Total	~850 €/mois